

# 토큰 생성 속도 향상을 위한 Speculative Decoding과 LLM을 위한 Diffusion 다음 토큰을 기다리지 않는 방법

이준범  
래블업(Lablup)



# CONTENTS

01 LLM은 왜 느릴까?

02 Speculative Decoding

03 Diffusion Language Model

04 지금 오픈소스는





# 01

## LLM은 왜 느릴까?



# GPU는 1초 수조번 계산, 답변은 1토큰



GPU 연산

초당 수조 번 이상

화면에 나오는 토큰

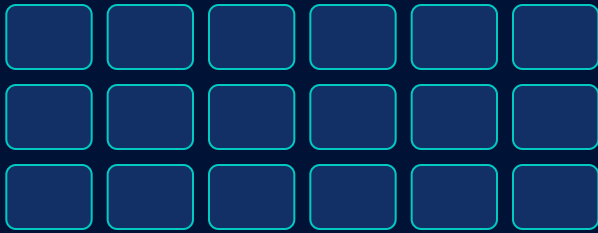
초당 수십 개

왜 이렇게 차이가 날까?



# 지연시간 두 종류: Prefill & Decode

프롬프트 토큰 N개



Prefill  
한 번에 병렬로

TTFT: 첫 토큰이 나오기까지

A

B

C

D

E

Decode  
토큰 1개

Decode  
토큰 1개

Decode  
토큰 1개

Decode  
토큰 1개

Decode  
토큰 1개

TPOT: 토큰 사이의 간격

× 출력 길이만큼 반복

## Prefill

### Compute-bound

프롬프트 전체를 한 번에 계산

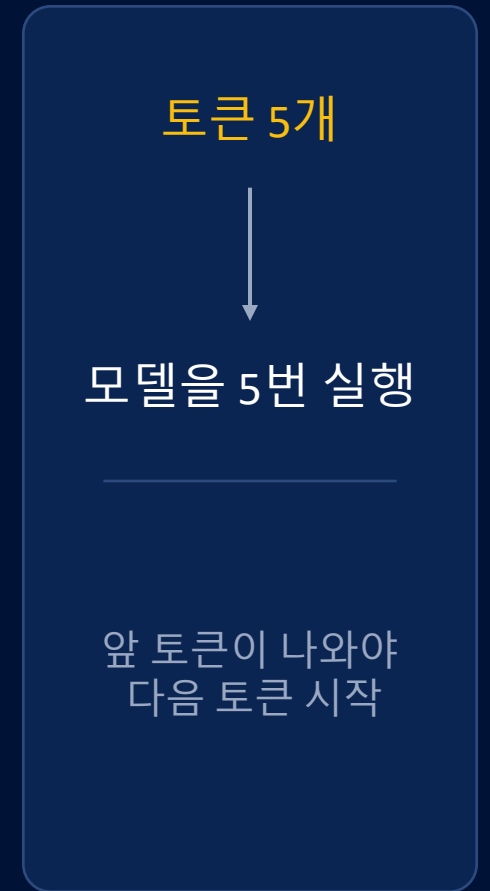
## Decode

### Memory-bound

스텝마다 전체 가중치를 메모리에서 다시 읽음

# 01

## 다음 토큰 = 앞 토큰이 나와야 생성

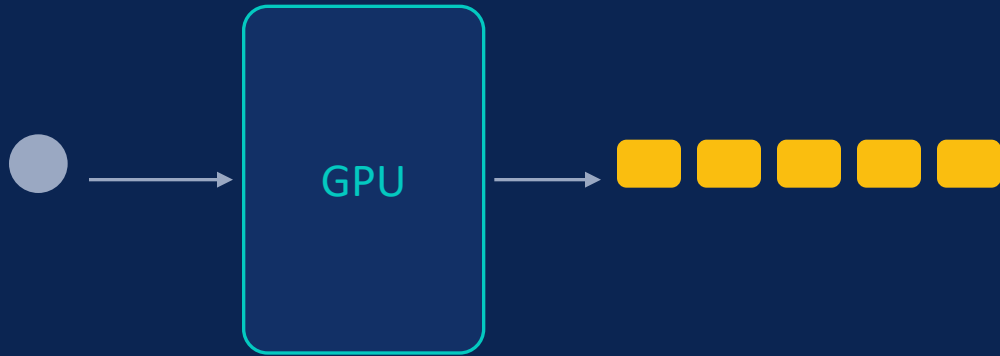


■ 이미 만든 토큰 ■ 이번 스텝에 만드는 토큰 □ 아직 만들지 않은 토큰

# 01

## GPU 늘리면 처리량은 증가, 1인당 속도는 그대로

사용자 1명



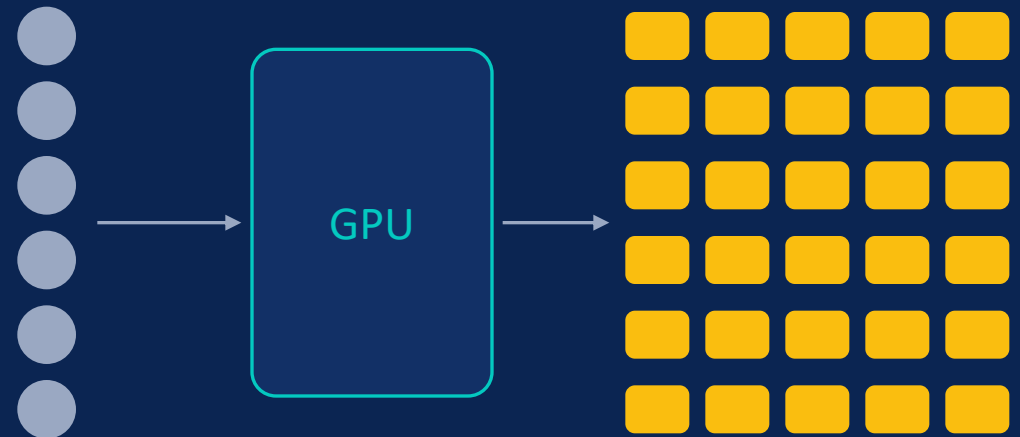
전체 TPS

1×

1인당 속도

1×

사용자 6명: 배치 처리



전체 TPS

≈4×+ ↑

1인당 속도

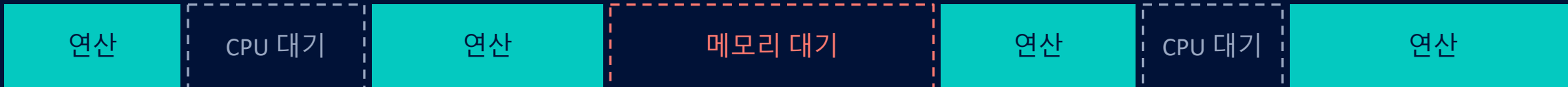
≈0.X×

한 사람에게 더 빨리 답하려면? → Speculative Decoding



# 한정된 GPU + 유휴 시간 줄이기

GPU 타임라인



점선 구간: GPU가 쉬는 시간

## CPU 대기

커널 실행/스케줄링 오버헤드

Fused GEMM

Kernel Fusion

CUDA Graph

MegaKernel

## 메모리 대기

가중치/KV cache를 읽어오는 시간

Quantize

Async

Batched

Speculative



# 조금 더 빠른 토큰 생성, 여전히 1번에 1토큰

## Quantization

한 번의 계산을 싸게

## Kernel Fusion

한 번의 계산을 빠르게

## Batching

여러 요청을 한꺼번에

최적화 전

A

B

C

D

E

5스텝

최적화 후

A

B

C

D

E

5스텝

스텝은 짧아졌지만

한 응답은 여전히 한 토큰씩

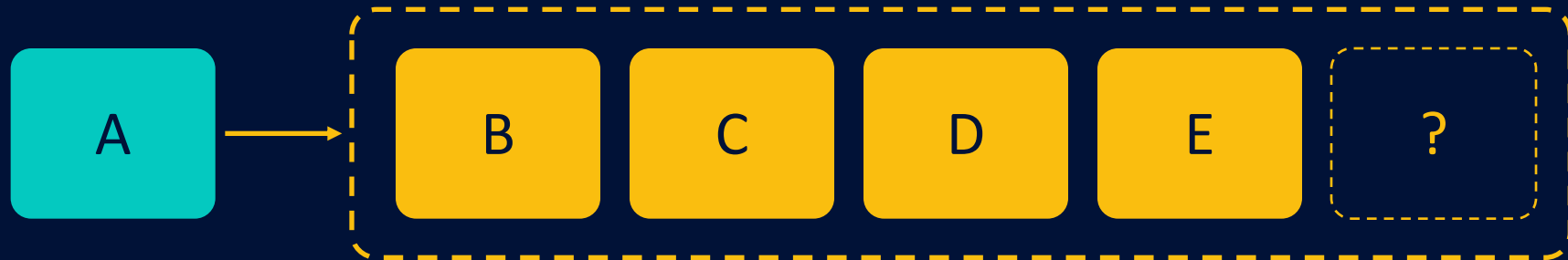
# 01

## 여러 토큰을 미리 만들어 본다?

지금



미리 추측하면?



한 번에 여러 토큰

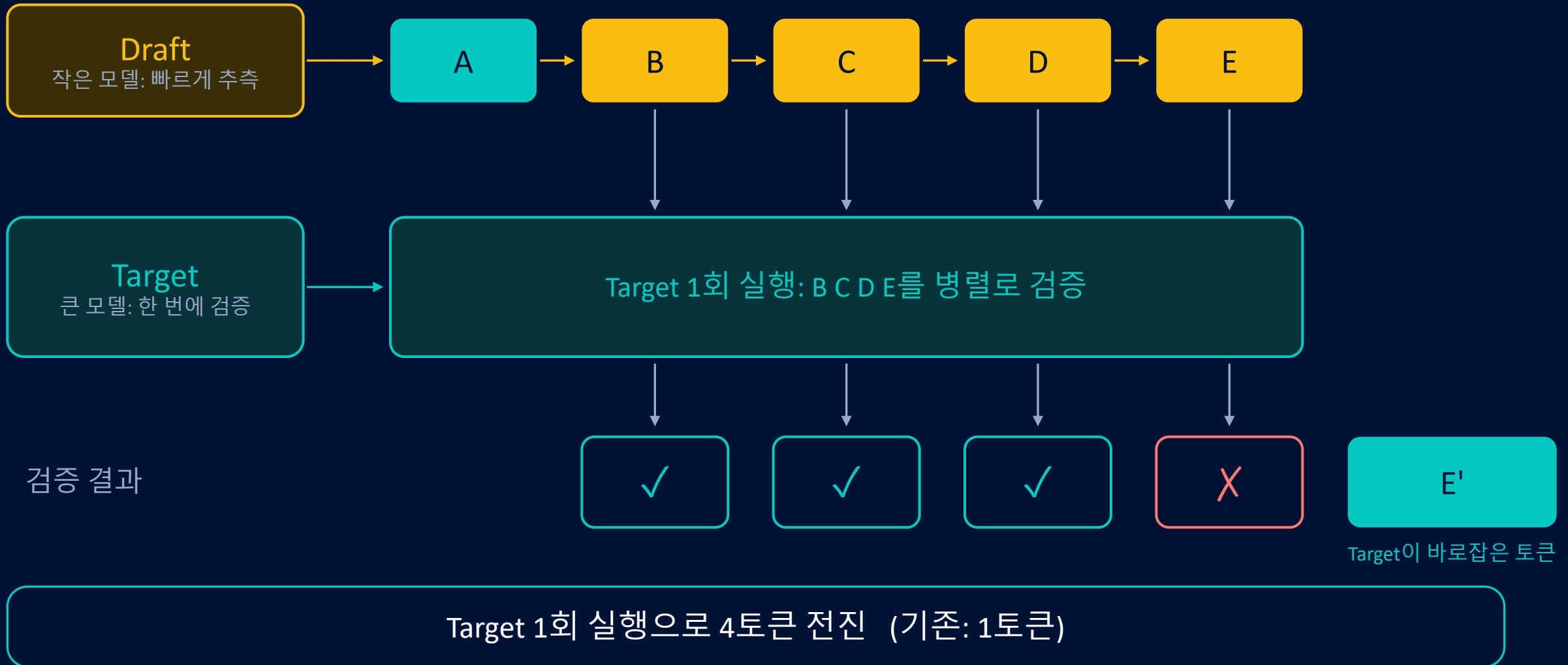


02

# Speculative Decoding

## 02

# 작은 모델이 추측 + 큰 모델이 일괄 검증



## 02

# 틀린 위치부터 버림. 결과는 Target과 동일

|           |      |        |       |    |        |       |
|-----------|------|--------|-------|----|--------|-------|
| Draft     | Open | source | makes | AI | faster | today |
| Target    | Open | source | makes | AI | more   |       |
| 판정        | ✓    | ✓      | ✓     | ✓  | ✗      | 버림    |
| 틀리면 바로 버림 |      |        |       |    |        |       |
| 최종 출력     | Open | source | makes | AI | more   | ...   |

Drafter는 말 그대로 초안만 만듦 → 최종 생성 텍스트는 Target 모델과 똑같은 결과



## 속도는 Draft 비용 x 적중률

### Draft 비용

만드는 비용이 낮을수록

×

### Acceptance Rate

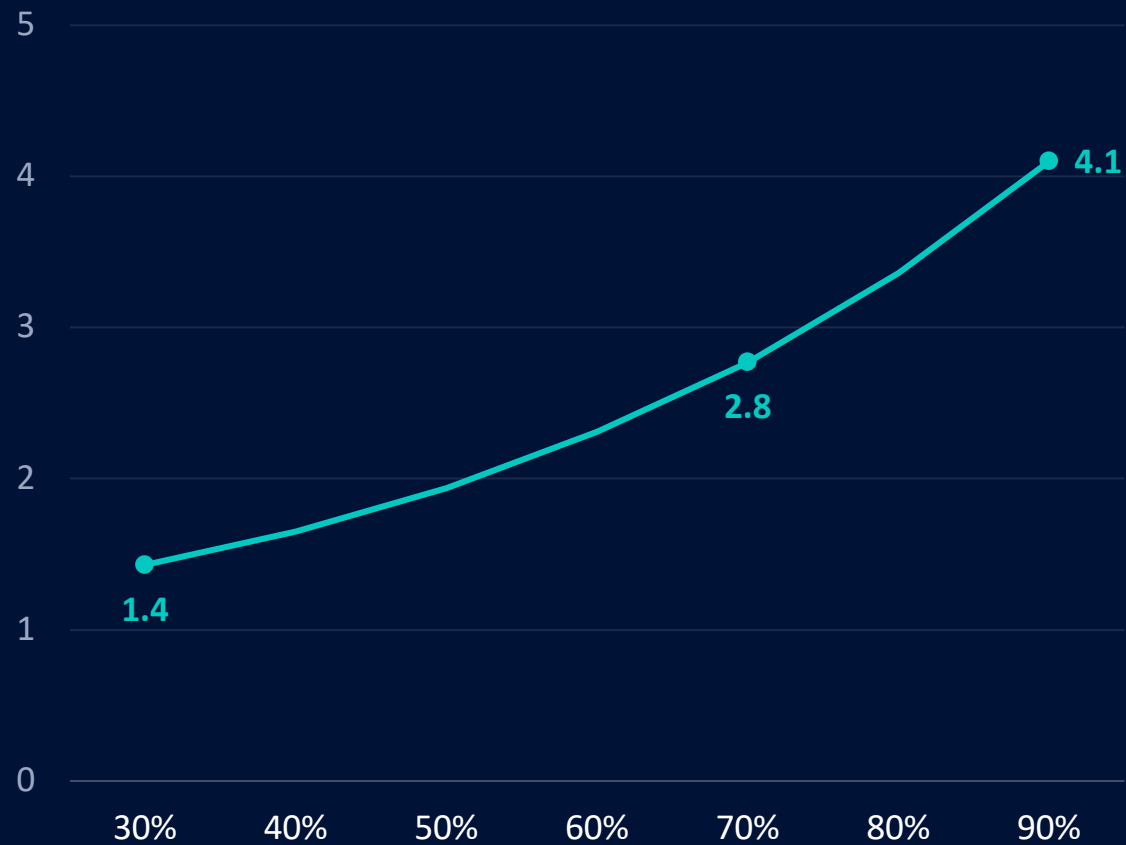
많이 맞힐수록

×

### 검증 길이

한 번에 많이 검증할수록

Target 1회당 전진 토큰 수, Acceptance Rate 별 (추측 길이 4)



기대값:  $(1-\alpha^5)/(1-\alpha)$ , Draft 비용 제외 / 출처: Leviathan et al., 2023



## 다양한 Speculative Decoding 방법

미래 토큰 후보를 어떻게 싸게 만들까?



Draft Model

작은 LLM을 별도로 사용



Self-Speculative

같은 모델의 일부 층만 사용



Medusa / EAGLE

Target 위에 가벼운 헤드 추가



N-gram Lookup

프롬프트에서 그대로 복사



Tree Speculation

여러 후보를 트리로 동시에 검증



Diffusion Drafter

블록을 한 번에 생성 (DFlash)



# Speculative Decoding도 순서는 그대로

Target 실행 횟수

AR  
12회



Spec  
4회



왼쪽에서 오른쪽으로 가는 순서는 그대로..

순서 자체를 바꿀 수는 없을까? → Diffusion LM



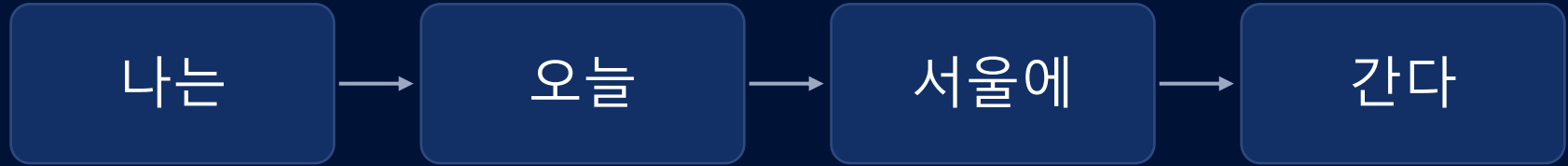
03

# Diffusion Language Model

# 03

## 꼭 왼쪽부터 차례대로 만들어야 할까?

지금까지



꼭 왼쪽부터여야 할까?



이렇게 시작한다면?



모든 위치를 동시에!

# 03

## 텍스트도 이미지처럼 여러 위치를 동시에

이미지 Diffusion



텍스트 Diffusion (Masked)



# 03

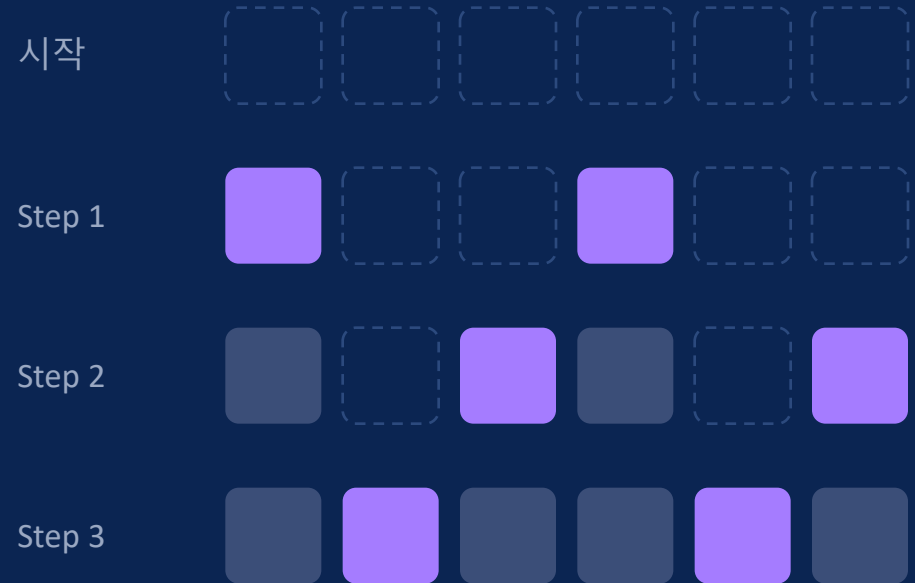
## Diffusion LM은 한 번에 여러 위치 생성

### Autoregressive



6토큰: 6스텝

### Diffusion LM



■ 이번 스텝에 채운 위치

6토큰: 3스텝



## Diffusion 확정은 N번 Forward의 결과

100토큰 응답을 만들 때 (개념도, 비율은 예시)

Autoregressive



~100스텝

스텝은 많지만 한 스텝은 가볍다 (KV cache)

Diffusion LM



N스텝

스텝은 적지만 스텝마다 전체 시퀀스를 본다

응답 시간  $\approx$  스텝 수  $\times$  스텝 비용

Diffusion  $\neq$  무조건 빠름



# Speculative vs Diffusion

## Sequential Dependency

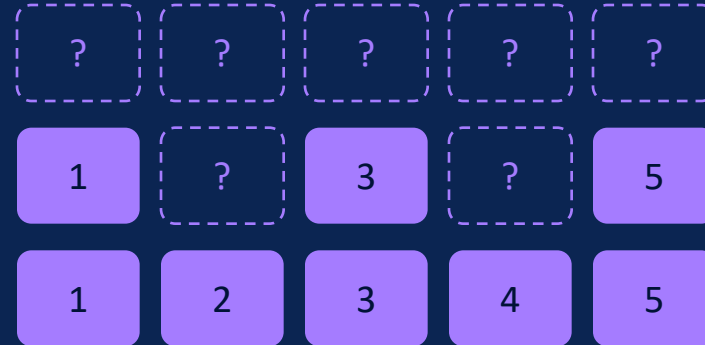
1 → 2 → 3 → 4 → 5

### Speculative Decoding



추측 → 검증

### Diffusion LM



반복 복원  
aka Denoising

순차 생성은 유지하면서  
여러 토큰을 한번에

순차 생성을  
반복 + 병렬 생성으로



04

지금 오픈소스는



# LLM 속도는 모델부터 ~ 하드웨어까지

Model

Llama / Qwen / DeepSeek / LLaDA

Draft 헤드 / Diffusion LM

Serving Engine

vLLM / SGLang / llama.cpp / mlxcel

Spec decoding / Batching

Runtime

PyTorch / MLX

Scheduling / KV cache

Kernel

Triton / FlashAttention / FlashInfer

Fused GEMM / Attention

Hardware

NVIDIA / AMD / Apple Silicon

메모리 대역폭

# 04

## 주요 서빙 엔진, 이미 다양한 가속 기법 지원

|                       | vLLM | SGLang | llama.cpp | mlxcel |
|-----------------------|------|--------|-----------|--------|
| Quantization          | ●    | ●      | ●         | ●      |
| Continuous Batching   | ●    | ●      | ●         | ●      |
| Prompt / Prefix Cache | ●    | ●      | ●         | ●      |
| Speculative Decoding  | ●    | ●      | ●         | ●      |
| EAGLE 계열 Draft 헤드     | ●    | ●      | ●         | ○      |
| Diffusion LLM 서빙      | ●    | ●      | ○         | ●      |

● 지원 ○ 미지원/실험적 / 2026년 9월 각 프로젝트 공식 문서 기준 / Diffusion LLM: vLLM·mlxcel은 DiffusionGemma, SGLang은 LLaDA 2.0 / llama.cpp Diffusion은 CLI 예제만



# 04 mlxcel: 로딩부터 추론까지 단일 네이티브 프로세스

일반적인 Python 스택

HTTP 서버 프로세스

Python 인터프리터

mlx-lm / 모델 코드

MLX (C++ / Metal)

여러 계층 / 여러 프로세스

OpenAI 호환

API 서버

80+

모델 아키텍처 지원

mlx-community

체크포인트를 변환 없이 로드

mlxcel

하나의 네이티브 프로세스 (Rust)

로딩

스케줄링

추론

Continuous Batching

Prompt Cache

Speculative

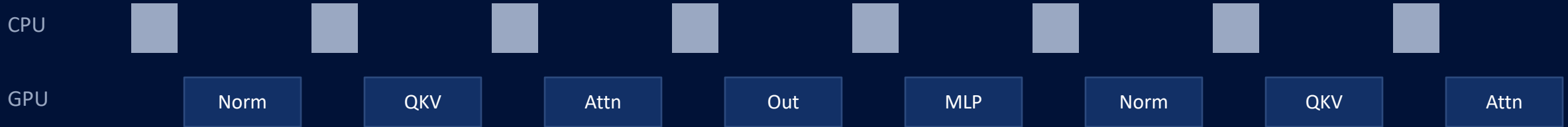


MLX C++ 바인딩: Apple Silicon / CUDA

# 04

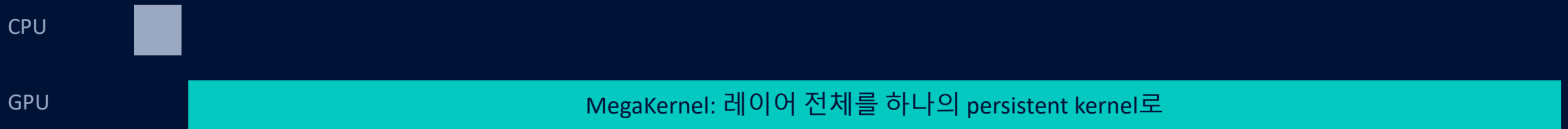
## 로컬 추론엔 여러 커널을 묶은 MegaKernel도 활용

여러 커널



커널마다 launch / 동기화 / 메모리 왕복 → GPU가 중간중간 쉰다

MegaKernel



launch 1회 / 중간 결과는 on-chip에 유지 → CPU 대기 제거

배치 크기가 1인 로컬 추론에서 효과가 가장 크다

# 오늘 이야기 정리

## Autoregressive

한 스텝에 한 토큰



## Speculative Decoding

추측하고 한 번에 검증



## Diffusion LM

여러 위치를 반복해서 복원



감사합니다.

